MongoDB Practical Assessment

Tools: MongoDB Compass + MongoDB Shell (`mongosh`)

Database: `commerce_assessment`

Part 1 — Database Setup

Create and use:

```
use commerce_assessment
```

Create a collection:

```
db.createCollection("products")
```

Insert the following data:

```
db.products.insertMany([
    {
        product_id: 101,
        product_name: "Ultra Laptop",
        category: "Electronics",
        brand: "TechPro",
        price: 72000,
        stock: 12,
        available: true,
        launch_date: ISODate("2026-01-15"),
        colors: ["Black", "Silver"],
        specifications: {
            ram: "16 GB",
            storage: "1 TB SSD",
            warranty_years: 2
        }
    },
    {
        product_id: 102,
```

```
    product_name: "Wireless Mouse",
    category: "Accessories",
    brand: "ClickMax",
    price: 1800,
    stock: 40,
    available: true,
    launch_date: ISODate("2026-02-10"),
    colors: ["Black", "Blue", "Red"],
    specifications: {
        wireless: true,
        warranty_years: 1
    }
},
{
    product_id: 103,
    product_name: "Office Desk",
    category: "Furniture",
    brand: "WorkSpace",
    price: 14500,
    stock: 8,
    available: true,
    launch_date: ISODate("2025-11-20"),
    colors: ["Brown", "Black"],
    specifications: {
        material: "Wood",
        adjustable: false,
        warranty_years: 3
    }
},
{
    product_id: 104,
    product_name: "Smart Monitor",
    category: "Electronics",
    brand: "VisionPlus",
    price: 26000,
    stock: 5,
    available: true,
    launch_date: ISODate("2026-03-01"),
    colors: ["Black"],
    specifications: {
        screen_size: "27 inch",
        resolution: "4K",
        warranty_years: 2
    }
```

```
    },
    {
        product_id: 105,
        product_name: "Gaming Chair",
        category: "Furniture",
        brand: "SeatPro",
        price: 18500,
        stock: 0,
        available: false,
        launch_date: ISODate("2026-01-05"),
        colors: ["Black", "Red"],
        specifications: {
            material: "Leather",
            adjustable: true,
            warranty_years: 3
        }
    },
    {
        product_id: 106,
        product_name: "Mechanical Keyboard",
        category: "Accessories",
        brand: "KeyWorks",
        price: 5200,
        stock: 18,
        available: true,
        launch_date: ISODate("2026-04-12"),
        colors: ["Black", "White"],
        specifications: {
            backlit: true,
            wireless: false,
            warranty_years: 1
        }
    }
])
```

Part 2 — CRUD Operations

Complete the following using `mongosh`.

1. Display all products.
2. Display only one product having `product_id = 103`.

3. Insert a new product with:
    - Product ID `107`
    - Product Name `Bluetooth Speaker`
    - Category `Electronics`
    - Brand `SoundMax`
    - Price `7500`
    - Stock `20`
    - Available `true`
4. Insert two additional products in a single operation.
5. Update the price of product `102` to `2000`.
6. Update both stock and availability of product `105`.
7. Increase the stock of product `104` by `10`.
8. Reduce the stock of product `106` by `3`.
9. Add a new field called:

```
discount_percent
```

with value `10` to product `101`.

10. Remove `discount_percent` from product `101`.
11. Delete one product using its `product_id`.
12. Delete all products where:

```
stock = 0
```

13. Verify the documents before and after each delete.

Part 3 — Filtering and Query Operators

14. Find all Electronics products.
15. Find products costing more than `₹10,000`.

16. Find products costing between `₹5,000` and `₹30,000`.

17. Find products whose stock is less than `10`.

18. Find products that are currently available.

19. Find products that are not available.

20. Find products belonging to either:

```
Electronics
Furniture
```

21. Find products that are not in the `Accessories` category.

22. Find products where:

```
price > 10000
AND
stock > 5
```

23. Find products where:

```
category = Electronics
OR
price < 5000
```

24. Find products launched after:

```
1 February 2026
```

25. Find products launched between:

```
1 January 2026
and
31 March 2026
```

Part 4 — Projection and Sorting

26. Display only:

```
product_name
price
```

Do not display `_id`.

27. Display:

```
product_name
category
stock
```

28. Sort products by price from lowest to highest.

29. Sort products by price from highest to lowest.

30. Sort products alphabetically by product name.

31. Display the three most expensive products.

32. Display the two products having the lowest stock.

Part 5 — Arrays

Use the existing `colors` field.

33. Find products available in `Black`.

34. Find products available in both:

```
Black
Silver
```

35. Find products available in either:

```
White
Blue
```

36. Add `Grey` to product `101`.

37. Add `Black` to product `101` without creating a duplicate if it already exists.

38. Remove `Red` from product `102` .

39. Display products having more than one color.

Part 6 — Embedded Documents

Use the existing:

```
specifications
```

embedded document.

40. Find products whose warranty is `2` years.

41. Find products where:

```
specifications.adjustable = true
```

42. Find products with:

```
specifications.wireless = true
```

43. Change the warranty of product `103` from `3` to `5` years.

44. Add:

```
specifications.country_of_origin
```

to any two products.

45. Find products manufactured in the country you added.

Part 7 — Data Types

46. Find documents where `price` is stored as a numeric type.

47. Find the datatype of:

```
launch_date
```

48. Find the datatype of:

```
colors
```

49. Find the datatype of:

```
specifications
```

50. Insert one intentionally incorrect product where:

```
price
```

is stored as a string instead of a number.

Example concept:

```
"25000"
```

Do not copy an existing product.

51. Write a query that identifies products where `price` is stored as a string.

52. Correct that product so `price` becomes numeric.

Part 8 — Customers Collection

Create:

```
db.createCollection("customers")
```

Insert:

```
db.customers.insertMany([
    {
        customer_id: 501,
        customer_name: "Aditya Rao",
        email: "aditya@email.com",
        active: true,
        address: {
            city: "Hyderabad",
            state: "Telangana",
            pincode: "500081"
        },
        interests: ["Electronics", "Accessories"]
    },
    {
        customer_id: 502,
        customer_name: "Maya Thomas",
        email: "maya@email.com",
        active: true,
        address: {
            city: "Chennai",
            state: "Tamil Nadu",
            pincode: "600028"
        },
        interests: ["Furniture", "Home"]
    },
    {
        customer_id: 503,
        customer_name: "Faisal Ahmed",
        email: "faisal@email.com",
        active: false,
        address: {
            city: "Hyderabad",
            state: "Telangana",
            pincode: "500034"
        },
        interests: ["Electronics"]
    },
    {
        customer_id: 504,
        customer_name: "Nikita Shah",
        email: "nikita@email.com",
        active: true,
        address: {
            city: "Mumbai",
```

```
            state: "Maharashtra",
            pincode: "400050"
        },
        interests: ["Furniture", "Electronics"]
    }
])
```

Complete the following:

53. Display all customers.
54. Find active customers.
55. Find customers from Hyderabad.
56. Find customers interested in Electronics.
57. Find customers interested in both:

```
Furniture
Electronics
```

58. Change customer `503` from inactive to active.
59. Change customer `504` 's city.
60. Add a new interest to customer `502` .

Part 9 — Array of Embedded Documents

Create a new collection:

```
db.createCollection("orders")
```

Insert:

```
db.orders.insertMany([
    {
        order_id: 9001,
        customer_id: 501,
        order_date: ISODate("2026-09-01"),
```

```
    status: "Delivered",
    items: [
        {
            product: "Ultra Laptop",
            quantity: 1,
            price: 72000
        },
        {
            product: "Wireless Mouse",
            quantity: 2,
            price: 2000
        }
    ]
},
{
    order_id: 9002,
    customer_id: 502,
    order_date: ISODate("2026-09-03"),
    status: "Processing",
    items: [
        {
            product: "Office Desk",
            quantity: 1,
            price: 14500
        }
    ]
},
{
    order_id: 9003,
    customer_id: 503,
    order_date: ISODate("2026-09-05"),
    status: "Delivered",
    items: [
        {
            product: "Smart Monitor",
            quantity: 2,
            price: 26000
        },
        {
            product: "Mechanical Keyboard",
            quantity: 1,
            price: 5200
        }
    ]
```

```
    }
])
```

Complete:

61. Display orders having status `Delivered`.

62. Find orders containing `Ultra Laptop`.

63. Find orders containing an item with quantity greater than `1`.

64. Find orders containing:

```
product = Smart Monitor
AND
quantity = 2
```

where both conditions belong to the same item.

65. Add another item to order `9002`.

66. Change order `9002` status to `Delivered`.

67. Remove one item from an order.

Part 10 — Document Design

For each requirement below, decide whether you would use:

```
Simple Field
Array
Embedded Document
Array of Embedded Documents
Separate Collection / Reference
```

68. Product can have five colors.

69. Customer has one residential address.

70. Order contains multiple products.

71. Product has RAM, storage and processor information.

72. Customer can have thousands of orders over many years.
73. Product can have millions of customer reviews.
74. Employee has first name and last name.
75. Product has multiple tags.

For each one, write **one short reason** for your design choice.

Final Challenge

Create a new collection called:

```
suppliers
```

Design at least **five supplier documents** yourself.

Each supplier must contain:

- Supplier ID
- Supplier Name
- City
- Active status
- At least two product categories supplied
- Contact information as an embedded document
- At least two warehouse locations as an array
- Registration date as a Date

Then perform the following:

76. Display all active suppliers.
77. Find suppliers from a city of your choice.
78. Find suppliers handling `Electronics`.
79. Find suppliers registered after `1 January 2026`.
80. Add another warehouse to one supplier.
81. Change the active status of one supplier.
82. Remove one category from one supplier.

83. Display supplier name and city only.

84. Sort suppliers by registration date.

85. Explain briefly why you used:

- an array for categories,
- an embedded document for contact details,
- an array for warehouse locations.